

Equalização de Histograma e Processamento Paralelo em Java

Sistemas Multinúcleo e Distribuídos

Mestrado em Engenharia Informática – 2025/2026

Porto, maio de 2026

Luis Manuel Nazário Mendes Santos

Versão 3, 2026-05-11

Conteúdo

Lista de Figuras	iv
Lista de Tabelas	v
Declaração de Integridade	ix
1 Introdução	1
2 Objetivos	3
3 Implementações	5
3.1 Solução Sequencial	5
3.2 Multithreaded sem Thread Pool	6
3.3 Thread Pool com ExecutorService	7
3.4 Fork/Join Framework	7
3.5 CompletableFuture	8
3.6 Tuning do Garbage Collector	9
3.6.1 Garbage Collectors Avaliados	9
3.6.2 Resultados	10
3.6.3 Análise e Escolha	11
4 Concorrência e Sincronização	13
4.1 Estrutura das Fases	13
4.2 Thread Safety	13
4.3 Estruturas de Dados e Mecanismos	14
5 Análise de Performance	15
5.1 Configuração do Ambiente	15
5.2 Resultados Globais (imagem 860×1276)	16
5.3 Escalabilidade por Dimensão de Imagem	17
5.4 Overhead de Abstração e Memória	20
5.5 Bottlenecks Identificados	20

6	Conclusões	23
6.1	Síntese dos Resultados	23
6.2	Comparação das Implementações	24
6.3	Garbage Collector	24
6.4	Reflexões do Projeto	24
	Referências	25

Lista de Figuras

1.1	Efeito visual da equalização de histograma (imagem 860×1276)	1
3.1	Tempo sequencial e melhor paralelo por Garbage Collector	10
3.2	Estatísticas de pausa GC por coletor (benchmark completo)	11
5.1	Speedup por número de threads (imagem 860×1276)	17
5.2	Tempo sequencial em função da dimensão da imagem	18
5.3	Melhor speedup por implementação e dimensão de imagem	19
5.4	Tempo de execução por número de threads (imagem 1920×1080)	20

Lista de Tabelas

3.1	Comparação de GCs — imagem 1920×1080 (-Xms256m -Xmx2g)	10
4.1	Mecanismos de sincronização por implementação	14
5.1	Benchmark completo — imagem 860×1276 (1 097 360 píxeis)	16
5.2	Escalabilidade: tempo sequencial e melhor speedup por dimensão	18
6.1	Comparação qualitativa das implementações	24

Declaração de Integridade

Declaro, por este meio, que o presente trabalho académico foi realizado com integridade.

Afirmo não ter recorrido a plágio, falsificação de resultados ou qualquer outra forma de utilização indevida de informação no decurso da sua elaboração.

Assim, o trabalho aqui apresentado é original e da minha autoria, não tendo sido previamente utilizado para outros fins que não os explicitamente reconhecidos.

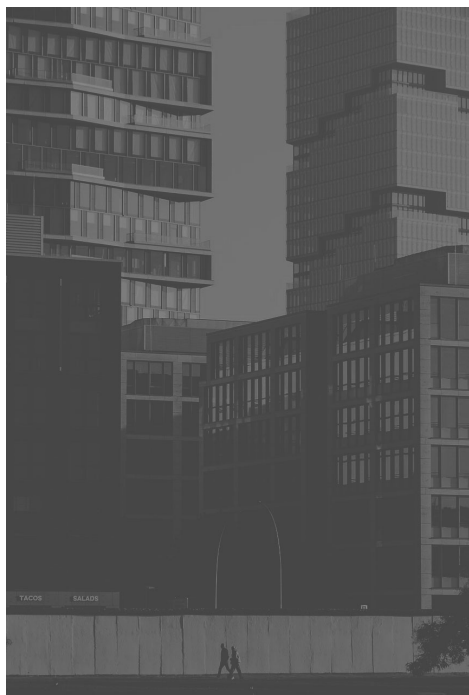
Confirmo ainda o meu conhecimento e respeito pelo Código de Conduta Ética do P.PORTO.

ISEP, Porto, 11 de maio de 2026

1 Introdução

O processamento digital de imagem é uma área com aplicações em medicina, fotografia, visão por computador e muitas outras disciplinas. Uma das operações mais fundamentais é a **equalização de histograma**, que redistribui as intensidades dos píxeis de modo a aumentar o contraste de imagens de baixo contraste.

Neste projeto, implementa-se a equalização de histograma em Java, explorando diferentes estratégias de concorrência: solução sequencial, multithreading manual, thread pools, o framework Fork/Join e o paradigma assíncrono com `CompletableFuture`. O objetivo não é apenas a correção dos resultados, mas também a análise de desempenho e escalabilidade de cada abordagem.



(a) Imagem original



(b) Imagem após equalização sequencial

Figura 1.1: Efeito visual da equalização de histograma (imagem 860×1276)

Os testes foram executados numa máquina com **22 processadores lógicos**. O con-

1 Introdução

junto de imagens de teste cobre quatro dimensões distintas para análise de escalabilidade: 512×512 (262 144 px), 860×1276 (1 097 360 px), 1920×1080 (2 073 600 px) e 3840×2160 (8 294 400 px).

O algoritmo de equalização é dividido em três fases:

1. **Histograma de luminosidade:** array de 256 inteiros onde a posição i contém o número de píxeis com luminosidade i .
2. **Histograma cumulativo:** array de 256 inteiros onde a posição i contém o número de píxeis com luminosidade $\leq i$.
3. **Reescrita dos píxeis:** cada píxel é transformado segundo:

$$\text{newLuminosity} = \left\lfloor 255 \times \frac{\text{cumulativeHist}[L]}{\text{totalPixels}} \right\rfloor$$

2 Objetivos

O projeto tem como objetivos principais:

- Implementar a equalização de histograma com diferentes abordagens de concorrência em Java.
- Comparar o desempenho (tempo de execução, consumo de memória, escalabilidade) de cada implementação.
- Explorar mecanismos de sincronização e garantir *thread safety* em todas as implementações paralelas.
- Configurar e analisar o impacto de diferentes *Garbage Collectors* da JVM no desempenho da aplicação.
- Documentar e justificar as escolhas de design e concorrência em cada abordagem.

A estrutura do projeto foi organizada em pacotes Java de acordo com a responsabilidade de cada componente:

- `sismd.filters` — Interface, classe base e todas as implementações do filtro.
- `sismd.benchmark` — Infraestrutura de medição de desempenho.
- `sismd.utils` — Utilitários de I/O de imagem (código fornecido pelo docente).
- `sismd.starter` — Código de demonstração original fornecido pelo docente.
- `sismd.Main` — Ponto de entrada com menu interativo.

3 Implementações

Todas as implementações partilham uma interface comum `HistogramEqualizer` e estendem a classe abstrata `AbstractFilter`, que disponibiliza os métodos utilitários `computeLuminosity`, `buildCumulativeHistogram` e `equalizedLuminosity`. Os métodos `computeHistogram` e `applyEqualization` da `SequentialFilter` são (*package-private*) e são reutilizados pelas implementações paralelas como operações de folha.

3.1 Solução Sequencial

A solução sequencial serve como baseline para comparação de desempenho. Processa o array de píxeis em três passagens sequenciais correspondentes às três fases do algoritmo.

Justificação: A implementação sequencial não tem overhead de criação de threads nem de sincronização, pelo que representa o custo mínimo do algoritmo. É indispensável para calcular o *speedup* das implementações paralelas e para verificar a correção dos resultados.

Listing 3.1: Núcleo da implementação sequencial

```
public Color[][] apply(Color[][] image) {
    Color[][] result = Utils.copyImage(image);
    int totalPixels = result.length * result[0].length;

    int[] hist      = computeHistogram(result, 0, result.length);
    int[] cumulative = buildCumulativeHistogram(hist);
    applyEqualization(result, 0, result.length, cumulative, totalPixels);

    return result;
}
```

3.2 Multithreaded sem Thread Pool

Esta implementação cria e gere threads manualmente, sem reutilização. A estratégia divide as linhas da imagem igualmente entre as threads disponíveis.

Estratégia de concorrência:

1. **Fase 1 (paralela):** Cada thread calcula um histograma parcial para as suas linhas, sem qualquer sincronização — não há estado partilhado durante esta fase.
2. **Barreira:** A thread principal aguarda a conclusão de todas as threads com `join()` e faz o merge dos histogramas parciais (256 iterações, custo desprezável).
3. **Fase 2 (sequencial):** O histograma cumulativo é calculado sequencialmente ($O(256)$).
4. **Fase 3 (paralela):** Cada thread reescreve os seus píxeis — as threads acedem a linhas disjuntas, pelo que não há necessidade de sincronização.

Justificação: A ausência de *locks* ou variáveis atómicas durante as fases paralelas elimina o overhead de sincronização. A opção por histogramas parciais por thread (em vez de um array partilhado com `AtomicIntegerArray`) evita falsa partilha de cache e contention.

Listing 3.2: Fase paralela de histograma com threads manuais

```
for (int t = 0; t < actualThreads; t++) {
    final int id = t;
    final int start = id * rows / actualThreads;
    final int end = (id + 1) * rows / actualThreads;
    threads[t] = new Thread(() ->
        partialHists[id] = SequentialFilter.computeHistogram(result,
            start, end)
    );
    threads[t].start();
}
joinAll(threads); // barreira: aguarda todas as threads

int[] hist = new int[256];
for (int[] partial : partialHists)
    for (int i = 0; i < 256; i++) hist[i] += partial[i];
```

3.3 Thread Pool com ExecutorService

Esta implementação substitui a criação manual de threads por um pool fixo gerido pelo `ExecutorService`. As threads são reutilizadas entre as fases 1 e 3, eliminando o custo de criação e destruição de threads.

Justificação: Em cenários com muitas execuções do filtro (como o benchmark), o overhead de criar e destruir threads a cada execução pode ser significativo. O thread pool amortiza esse custo. As tarefas de histograma retornam `Future<int[]>`, permitindo recolher os resultados de forma limpa sem variáveis partilhadas.

Listing 3.3: Submissão de tarefas ao thread pool

```
List<Future<int[]>> histFutures = new ArrayList<>(actualThreads);
for (int t = 0; t < actualThreads; t++) {
    final int start = t * rows / actualThreads;
    final int end = (t + 1) * rows / actualThreads;
    histFutures.add(pool.submit(() ->
        SequentialFilter.computeHistogram(result, start, end)
    ));
}
int[] hist = new int[256];
for (Future<int[]> f : histFutures) {
    int[] partial = f.get();
    for (int i = 0; i < 256; i++) hist[i] += partial[i];
}
```

3.4 Fork/Join Framework

A implementação Fork/Join utiliza o paradigma de *divide-and-conquer* para dividir recursivamente o array de linhas até atingir um limiar (`THRESHOLD = 32` linhas), a partir do qual a computação é sequencial.

Estratégia:

- `HistogramTask (RecursiveTask<int[]>)`: divide o intervalo de linhas ao meio; a metade esquerda é bifurcada (`fork`), a metade direita é computada na thread atual (`compute`), e depois faz-se `join + merge`.
- `EqualizationTask (RecursiveAction)`: usa `invokeAll` para executar ambas as metades em paralelo.

Justificação: O ForkJoinPool usa *work stealing*: threads que ficam sem tarefas roubam tarefas de outras, maximizando a utilização dos processadores. O limiar de 32 linhas foi escolhido de modo a criar tarefas granulares suficientes para manter todas as threads ocupadas sem excessivo overhead de *forking*.

Listing 3.4: HistogramTask com Fork/Join

```
@Override
protected int[] compute() {
    if (end - start <= THRESHOLD)
        return SequentialFilter.computeHistogram(image, start, end);

    int mid = (start + end) >>> 1;
    HistogramTask left = new HistogramTask(image, start, mid);
    HistogramTask right = new HistogramTask(image, mid, end);

    left.fork(); // execucao assincrona
    int[] rightResult = right.compute(); // execucao sincrona
    int[] leftResult = left.join(); // aguarda left

    for (int i = 0; i < 256; i++) leftResult[i] += rightResult[i];
    return leftResult;
}
```

3.5 CompletableFuture

A implementação com CompletableFuture usa programação assíncrona declarativa para expressar a composição de tarefas sem gestão explícita de threads ou bloqueios.

Estratégia:

- `supplyAsync` lança as tarefas de histograma assincronamente num `ExecutorService` dedicado.
- `CompletableFuture.allOf(...).thenApply(...)` aguarda a conclusão de todas as tarefas e executa o merge como uma continuação não bloqueante.
- `runAsync` lança as tarefas de equalização; `allOf(...).join()` aguarda a conclusão.

Justificação: O CompletableFuture permite expressar dependências entre tarefas de forma declarativa. O uso de `thenApply` para o merge garante que este só ocorre após

todas as tarefas estarem completas, sem *polling* nem bloqueio prematuro. O `join()` sobre os futuros dentro do `thenApply` é não bloqueante porque todos já terminaram no momento em que `allOf` dispara a continuação.

Listing 3.5: Merge assíncrono com `allOf` e `thenApply`

```
int[] hist = CompletableFuture
    .allOf(histFutures.toArray(new CompletableFuture[0]))
    .thenApply(v -> {
        int[] merged = new int[256];
        for (CompletableFuture<int[]> f : histFutures) {
            int[] partial = f.join(); // nao bloqueia: todos ja concluíram
            for (int i = 0; i < 256; i++) merged[i] += partial[i];
        }
        return merged;
    })
    .join();
```

3.6 Tuning do Garbage Collector

O benchmark foi executado separadamente com quatro *Garbage Collectors* da JVM (`-Xms256m -Xmx2g`), usando a imagem 1920×1080 como referência. Os logs de pausas foram capturados com `-Xlog:gc:<ficheiro>.log`.

3.6.1 Garbage Collectors Avaliados

Serial GC (`-XX:+UseSerialGC`): Usa uma única thread de GC, com pausas *stop-the-world* longas (média 32 ms, máx. 69 ms). Surpreendentemente, produz o **menor tempo de execução** neste benchmark (43 ms sequencial, 8 ms melhor paralelo) porque não tem threads concorrentes a competir com a aplicação nem *write barriers* complexas.

Parallel GC (`-XX:+UseParallelGC`): Múltiplas threads de GC reduzem a pausa média para 12.6 ms (máx. 40.9 ms). Tempo de execução comparável ao Serial GC (41 ms sequencial). Optimizado para throughput, à custa de maior variância nas pausas.

G1GC (`-XX:+UseG1GC`): Divide a heap em regiões e recolhe as mais rentáveis primeiro. Pausas curtas e previsíveis (média 7.9 ms, máx. 10.8 ms). Contudo, o overhead da gestão de regiões e das *write barriers* aumenta o tempo sequencial para 62 ms (+44% face ao Serial GC).

ZGC (-XX:+UseZGC): Recolha maioritariamente concorrente — sem pausas *stop-the-world* significativas. No entanto, os threads de marcação concorrentes disputam os 22 núcleos com a aplicação, elevando o tempo sequencial para 71 ms (+65% face ao Serial GC) e a memória consumida para 112–132 MB.

3.6.2 Resultados

Tabela 3.1: Comparação de GCs — imagem 1920×1080 (-Xms256m -Xmx2g)

GC	Seq. (ms)	Melhor (ms)	Speedup máx.	Eventos GC	Pausa média	Pausa máx.
Serial GC	43	8	5.38×	121	32.0 ms	69.1 ms
Parallel GC	41	10	4.10×	176	12.6 ms	40.9 ms
G1GC	62	24	2.58×	105	7.9 ms	10.8 ms
ZGC	71	26	2.73×	≈0	≈0 ms	≈0 ms

As Figuras 3.1 e 3.2 ilustram graficamente os dados acima.

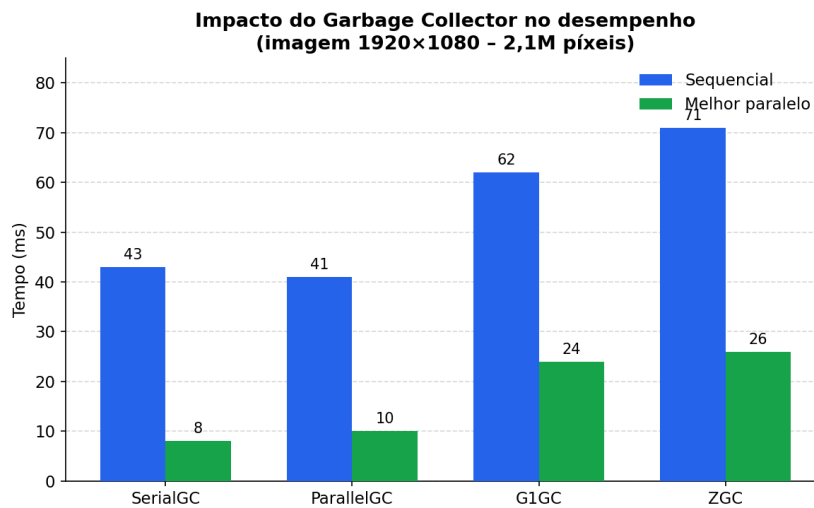


Figura 3.1: Tempo sequencial e melhor paralelo por Garbage Collector

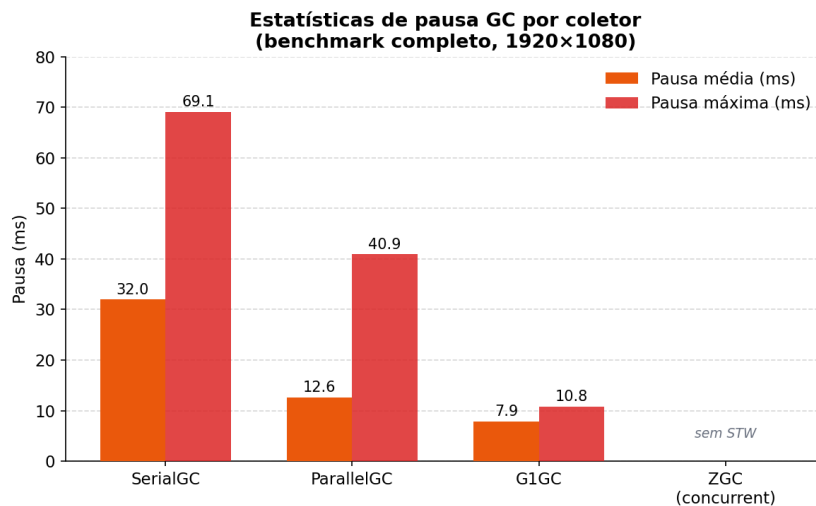


Figura 3.2: Estatísticas de pausa GC por coletor (benchmark completo)

3.6.3 Análise e Escolha

O resultado mais contraintuitivo é que o **SerialGC produz o melhor throughput** apesar de ter as pausas mais longas. A razão é que as pausas ocorrem entre medições (no `rt.gc()` explícito do benchmark), enquanto o que afeta o tempo medido é o overhead do GC durante a execução do filtro: *write barriers*, alocação de regiões e threads de fundo concorrentes. O Serial GC não tem nenhum destes.

O **ZGC** é o polo oposto: zero pausas STW, mas threads de marcação concorrentes que disputam os 22 núcleos com a aplicação — um custo desproporcionado para uma carga de trabalho *compute-heavy* e de curta duração.

Recomendação:

- **Processamento batch (máximo throughput):** Serial GC ou Parallel GC.
- **Serviço interativo (latência previsível):** G1GC com `-XX:MaxGCPauseMillis=100`.
- **Serviço de baixa latência com heap grande:** ZGC (não adequado para este workload).

Para este projeto, adota-se **G1GC** como configuração de referência por ser o padrão do JDK moderno e oferecer pausas previsíveis aceitáveis para uso interativo:

```
-XX:+UseG1GC -XX:MaxGCPauseMillis=200
-XX:G1HeapRegionSize=4m -Xms256m -Xmx2g
```


4 Concorrência e Sincronização

4.1 Estrutura das Fases

O algoritmo de equalização de histograma tem uma estrutura de fases (*pipeline barrier*):

1. **Fase 1** (paralelizável): Computação do histograma — os píxeis são independentes.
2. **Barreira**: Merge dos histogramas parciais — necessita que todas as threads da fase 1 tenham terminado.
3. **Fase 2** (sequencial): O histograma cumulativo é calculado em $O(256)$ — custo negligenciável.
4. **Fase 3** (paralelizável): Reescrita dos píxeis — independentes e em linhas disjuntas.

4.2 Thread Safety

Fase 1 (histograma): Cada thread opera sobre um array local `int[256]`. Não há estado compartilhado mutável. Após `join()/Future.get()/allOf().join()`, as escritas de todas as threads são visíveis na thread principal (garantia de *happens-before* da API de concorrência Java).

Merge: Executado exclusivamente na thread principal. Não requer sincronização.

Fase 3 (equalização): Cada thread escreve em linhas disjuntas do array `result`. O array `cumulative` é lido mas nunca escrito pelas threads, pelo que não há *data races*.

4.3 Estruturas de Dados e Mecanismos

Tabela 4.1: Mecanismos de sincronização por implementação

Implementação	Sincronização	Justificação
Multithreaded	<code>Thread.join()</code>	Barreira explícita entre fases
Thread Pool	<code>Future.get()</code>	Bloqueante por tarefa; recolhe resultado
Fork/Join	<code>fork/join, invokeAll</code>	Implícito no framework
CompletableFuture	<code>allOf().join()</code>	Composição declarativa de dependências

A opção por histogramas parciais por thread (sem `synchronized` nem `AtomicIntegerArray`) é intencional: elimina *contention* e falsa partilha de cache. O custo do merge (256 adições) é desprezável face ao trabalho paralelo.

5 Análise de Performance

5.1 Configuração do Ambiente

- **JVM:** Java 21, sem flag explícita de GC (secção 3.6)
- **Imagens de teste:** 512×512, 860×1276, 1920×1080, 3840×2160
- **Processadores:** 22 lógicos (`Runtime.getRuntime().availableProcessors()`)
- **Threads testadas:** 1, 2, 4, 8, 22
- **Metodologia:** 3 execuções de aquecimento (JIT) + média de 5 medições com `System.nanoTime()`

5.2 Resultados Globais (imagem 860×1276)

Tabela 5.1: Benchmark completo — imagem 860×1276 (1 097 360 píxeis)

Implementação	Tempo (ms)	Memória (MB)	Speedup
Sequential	33	40.29	1.00×
Multithreaded – 1 thread	24	39.92	1.38×
Multithreaded – 2 threads	16	40.32	2.06×
Multithreaded – 4 threads	9	40.36	3.67×
Multithreaded – 8 threads	6	42.00	5.50×
Multithreaded – 22 threads	7	54.05	4.71×
Thread Pool – 1 thread	24	38.04	1.38×
Thread Pool – 2 threads	14	38.50	2.36×
Thread Pool – 4 threads	9	39.65	3.67×
Thread Pool – 8 threads	9	39.44	3.67×
Thread Pool – 22 threads	7	40.01	4.71×
Fork/Join	7	39.30	4.71×
CompletableFuture – 1 thread	23	38.04	1.43×
CompletableFuture – 2 threads	14	38.76	2.36×
CompletableFuture – 4 threads	10	39.77	3.30×
CompletableFuture – 8 threads	7	39.12	4.71×
CompletableFuture – 22 threads	6	40.02	5.50×

O melhor speedup obtido é de **5.50×** (Multithreaded 8T e CompletableFuture 22T), partindo de 33 ms sequenciais até 6 ms. A Figura 5.1 ilustra a evolução do speedup com o número de threads para as três implementações paralelizáveis.

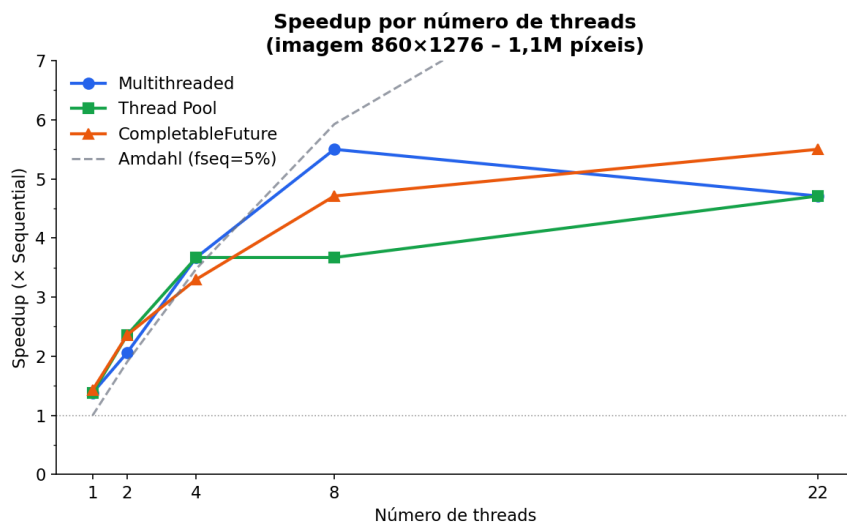


Figura 5.1: Speedup por número de threads (imagem 860×1276)

Nota-se que o Multithreaded sem pool atinge o pico em 8 threads e regride ligeiramente para 22, enquanto o CompletableFuture continua a ganhar até 22 threads, evidenciando que o overhead de gestão do pool de tarefas é amortizado de forma diferente em cada abstração.

5.3 Escalabilidade por Dimensão de Imagem

A Figura 5.2 mostra que o tempo sequencial cresce aproximadamente de forma linear com o número de píxeis — o que é esperado, pois o algoritmo é $O(N)$.

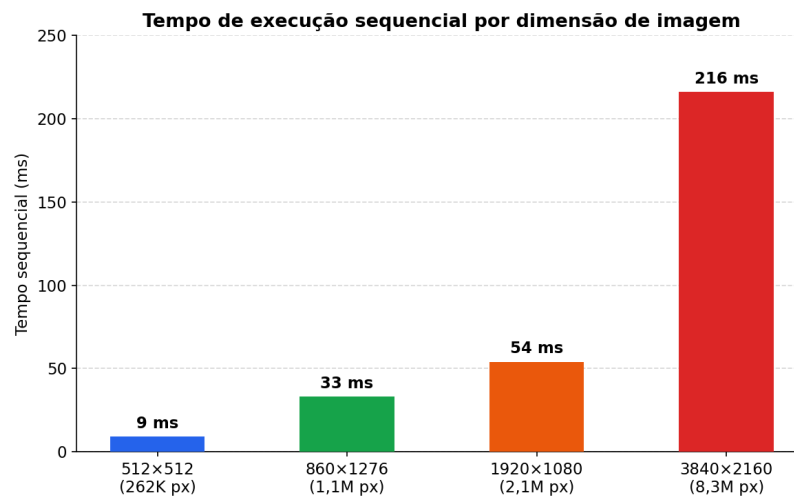


Figura 5.2: Tempo sequencial em função da dimensão da imagem

Tabela 5.2: Escalabilidade: tempo sequencial e melhor speedup por dimensão

Imagem	Píxeis	Seq. (ms)	Melhor (ms)	Speedup máx.
512×512	262 144	9	3	3.00× (TP/CF 8T)
860×1276	1 097 360	33	6	5.50× (MT 8T, CF 22T)
1920×1080	2 073 600	54	18	3.00× (MT 22T)
3840×2160	8 294 400	216	79	2.73× (TP 22T)

A Figura 5.3 compara o melhor speedup de cada implementação para as quatro dimensões:

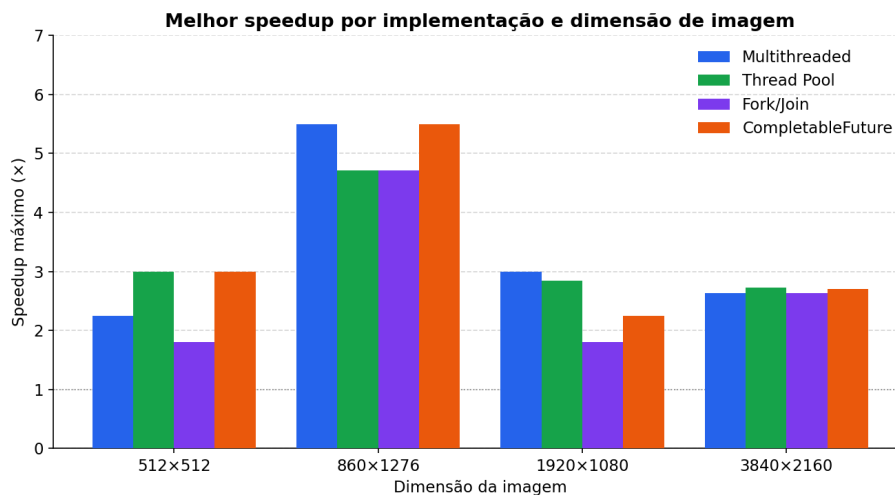


Figura 5.3: Melhor speedup por implementação e dimensão de imagem

Observa-se um padrão não-monotónico: o speedup **cresce** da imagem pequena para a média (512→860×1276), mas **decrece** para as imagens grandes. Isto explica-se por dois fenómenos opostos:

- **Imagens pequenas (512×512):** o overhead de threading é comparável ao trabalho útil — o speedup máximo é apenas 3× apesar de 8 threads disponíveis.
- **Sweet spot (1M píxeis):** o trabalho por thread é suficiente para amortizar os overheads; Multithreaded 8T e CF 22T atingem 5.50×.
- **Imagens grandes (4K, 8.3M px):** todas as implementações a 22 threads convergem para 79–82 ms. O bottleneck desloca-se da CPU para a **largura de banda de memória** — a Lei de Amdahl aplicada a recursos de hardware, não apenas ao código.

A Figura 5.4 detalha o comportamento para Full HD (2.1M píxeis), onde o Fork/Join degrada visivelmente:

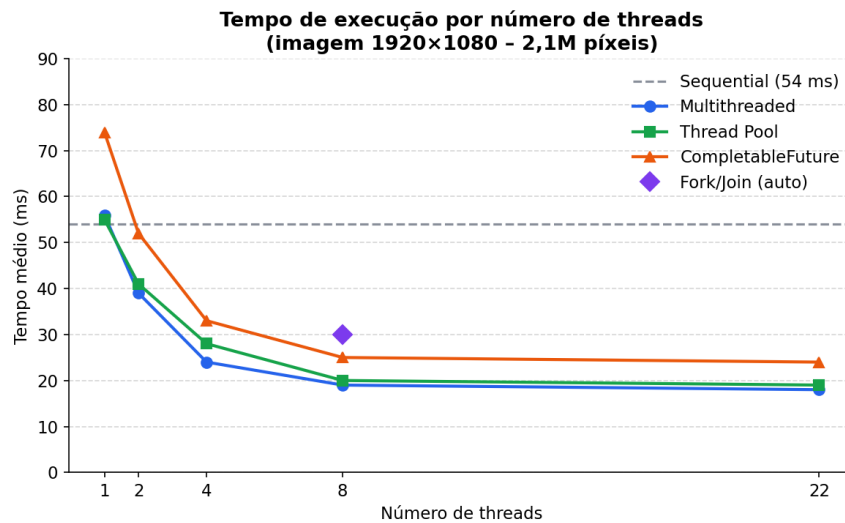


Figura 5.4: Tempo de execução por número de threads (imagem 1920×1080)

O Fork/Join apresenta 30 ms nesta imagem, contra 18 ms do Multithreaded manual. Com 1080 linhas e `THRESHOLD=32`, o framework cria $\lceil 1080/32 \rceil = 34$ tarefas-folha, sobrecarregando o *work-stealing scheduler* para uma granularidade que não justifica a divisão recursiva.

5.4 Overhead de Abstração e Memória

O `CompletableFuture` com 1 thread é consistentemente mais lento do que o `Sequential` em imagens grandes: 74 ms vs. 54 ms em Full HD (−37%), e 246 ms vs. 216 ms em 4K (−14%). O overhead de `supplyAsync`, `allOf` e `thenApply` não é desprezável quando o trabalho útil por tarefa é pequeno.

O consumo de memória mantém-se estável entre implementações (38–42 MB para a imagem 860×1276, 76 MB para Full HD, 301 MB para 4K), refletindo maioritariamente o tamanho da imagem. O Multithreaded a 22T apresenta 54 MB na imagem 860×1276 (+34% face ao Sequential) devido à criação de 22 arrays `int[256]` de histograma parcial — overhead de apenas 22 KB, mas que fica visível na medição por diferença de heap.

5.5 Bottlenecks Identificados

1. **Barreira sequencial (fase 2):** O histograma cumulativo ($O(256)$) é intrinsecamente sequencial e separa a fase 1 da fase 3, impedindo um pipeline contínuo.

2. **Saturação de memória em 4K:** A 8.3M píxeis e 22 threads, adicionar mais cores não reduz o tempo — todos os threads disputam o mesmo barramento de memória.
3. **Criação de objetos Color:** A fase 3 instancia um `Color` por píxel (até 8.3M objetos por execução), pressionando o GC de forma independente do grau de paralelismo. Substituir por arrays `int []` reduziria este custo.

6 Conclusões

6.1 Síntese dos Resultados

Este projeto demonstrou que a paralelização da equalização de histograma produz ganhos reais, mas limitados pela estrutura do algoritmo e pelos recursos de hardware. Os resultados principais são:

- O melhor speedup absoluto foi de **5.50×** (Multithreaded 8T e CompletableFuture 22T na imagem 860×1276, 33 ms → 6 ms) numa máquina de 22 processadores.
- O **sweet spot** da paralelização situa-se em imagens de 1M píxeis: grande o suficiente para amortizar o overhead de threading, pequeno o suficiente para caber em cache.
- Em imagens 4K (8.3M px), todas as implementações a 22T convergem para 79–82 ms, limitadas pela **largura de banda de memória**, não pela capacidade de processamento.
- O **CompletableFuture com 1 thread é mais lento que o Sequential** em imagens grandes (overhead de 14–37%), ilustrando que abstrações assíncronas têm custo real quando o trabalho por tarefa é pequeno.
- O **Fork/Join degrada em Full HD** (30 ms vs. 18 ms Multithreaded): o THRESHOLD=32 linhas cria granularidade excessiva para 1080 linhas, sobrecarregando o work-stealing scheduler.

6.2 Comparação das Implementações

Tabela 6.1: Comparação qualitativa das implementações

Implementação	Throughput	Escalabilidade	Complexidade	Overhead
Sequential	Baseline	N/A	Muito baixa	Nenhum
Multithreaded	Alto	Boa até 8T	Média	Baixo
Thread Pool	Alto	Boa até 22T	Média	Baixo
Fork/Join	Médio-alto	Variável	Alta	Médio
CompletableFuture	Alto	Boa até 22T	Média	Médio-alto (1T)

6.3 Garbage Collector

O resultado mais surpreendente foi que o **Serial GC produz o melhor throughput** para este workload (43 ms sequencial, 8 ms melhor paralelo), apesar de ter as pausas individuais mais longas. A causa é que o overhead do GC que afeta o tempo medido é diferente das pausas STW: write barriers, threads de fundo concorrentes e overhead de alocação — o Serial GC minimiza todos estes durante a execução da aplicação.

O **ZGC** é o pior para workloads compute-heavy de curta duração: os seus threads de marcação concorrentes disputam os 22 núcleos com a aplicação, penalizando +65% o tempo sequencial.

O **G1GC** é adotado como configuração de referência por ser o padrão do JDK moderno e oferecer pausas previsíveis adequadas para uso interativo.

6.4 Reflexões do Projeto

A **Lei de Amdahl** explica o comportamento observado: a fase do histograma cumulativo (O(256)) é intrinsecamente sequencial e constitui o teto teórico do speedup. Com uma fração sequencial estimada de 5%, o speedup máximo teórico seria 15×; o valor obtido de 5.5× indica que overhead adicional (coordenação, memória, GC) reduz o speedup real para 37% do teórico.

Referências

- [1] R.C. Gonzalez and R.E. Woods. *Digital Image Processing*, 4th edition. Pearson, 2018.
- [2] Robert Sedgewick and Kevin Wayne. *Computer Science: An Interdisciplinary Approach*, 1st edition. Addison-Wesley Professional, 2016.
- [3] Brian Goetz et al. *Java Concurrency in Practice*. Addison-Wesley, 2006.
- [4] Oracle Corporation. *Fork/Join Framework*. Java Documentation, 2024. <https://docs.oracle.com/javase/tutorial/essential/concurrency/forkjoin.html>
- [5] Oracle Corporation. *CompletableFuture API*. Java SE 21 Documentation, 2024. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CompletableFuture.html>